SQLite Tutorial 3: From a Wide Table to the EAV Model

Dr. Frank Xu

October 27, 2026

Introduction

A contact can have many attributes: a name, several phone numbers, several email addresses, and other values. We will begin with the most obvious design: one table containing one column for each attribute. After using that table, we will identify its limitations and redesign it so that each attribute is stored as a separate row. This second approach is called the **Entity–Attribute–Value (EAV) model**.

The example uses readable table and column names so that the relationships and design tradeoffs are easy to follow.

Learning objectives

After completing this tutorial, you should be able to:

- create a contact table that stores all attributes in one row;
- compare a wide-table design with an EAV design;
- explain the entity, attribute, and value parts of the EAV model;
- use identifier columns to connect related tables;
- insert multiple attributes for one contact;
- join three tables to reconstruct a readable contact; and
- search for individual contact attributes.

Step 1: Create and Open the Database

Open a terminal in the tutorial directory and create a database named `contacts.db`:

```
sqlite3 contacts.db
```

Make query results easier to read:

```
.headers on
.mode column
.nullvalue NULL
```

Step 2: Begin with One Contact Table

The simplest design places every attribute in the same row. We will call this table `contacts_flat` because its columns do not refer to other tables.

```
CREATE TABLE contacts_flat (
    contact_id   INTEGER PRIMARY KEY,
    account_name TEXT NOT NULL,
    name         TEXT NOT NULL,
    phone        TEXT,
    email        TEXT
);

INSERT INTO contacts_flat
    (contact_id, account_name, name, phone, email)
VALUES
    (101, 'Personal', 'John Smith',
     '+1 555-1234', 'john@gmail.com');
```

Querying the contact is straightforward because all of its values are in one row:

```
SELECT * FROM contacts_flat;
```

```
contact_id  account_name  name        phone        email
----------  ------------  ----------  -----------  --------------
101         Personal      John Smith  +1 555-1234  john@gmail.com
```

This design is easy to create and read when the possible attributes are fixed and each attribute occurs at most once per contact.

Step 3: Add a Second Phone Number

Suppose John has another phone number. The table has only one `phone` column, so there is no place to store it as a separate value. One possible fix is to add another column:

```
ALTER TABLE contacts_flat ADD COLUMN phone2 TEXT;

UPDATE contacts_flat
SET phone2 = '+1 555-9876'
WHERE contact_id = 101;

SELECT contact_id, name, phone, phone2
FROM contacts_flat;
```

```
contact_id  name        phone        phone2
----------  ----------  -----------  -----------
101         John Smith  +1 555-1234  +1 555-9876
```

This solves the immediate problem, but a third number would require `phone3`, and a fourth would require `phone4`. Contacts with only one number would leave the extra columns `NULL`. Adding addresses or other new attributes would require still more schema changes.

Storing several numbers in one text value, such as `+1 555-1234, +1 555-9876`, is also undesirable. SQLite would treat the combined value as one item, making individual numbers harder to search, update, or count. We therefore need a design that adds rows instead of columns.

The Entity–Attribute–Value (EAV) Design

The second approach is an EAV design. The **entity** is the contact in `contacts`. The **attribute** is the kind of information, such as `Name` or `Phone`, identified in `attributes`. The **value** is the actual name, phone number, or email address stored in `data.value`.

The EAV design uses the following tables:

Table	Important columns	Purpose
`contacts`	`contact_id`, `account_name`	Identifies a contact and its source account
`data`	`data_id`, `contact_id`, `attribute_id`, `value`	Stores one attribute per row
`attributes`	`attribute_id`, `attribute_name`	Defines each available attribute

Figure 1 shows the two one-to-many relationships. One contact can own zero or more data rows, and one attribute can describe zero or more data rows. For these relationships to remain meaningful, every `contact_id` and `attribute_id` in `data` should match a row in the corresponding table. This tutorial uses matching identifiers but does not add database constraints to enforce the matches.

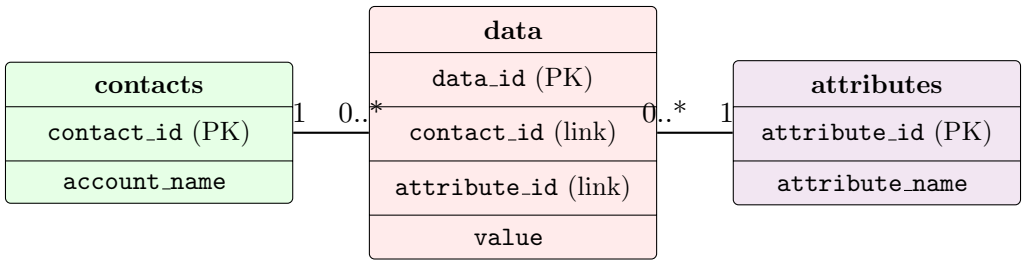


Figure 1: ER diagram for the EAV design. PK denotes a primary key, and the linking columns connect `data` to the other tables.

In the EAV design, `data.value` stores `John Smith`. The `attribute_id` column links this value to the `Name` row in `attributes`. A second phone number requires one more row rather than a new column.

Step 4: Create the EAV Tables

Create the lookup table first, followed by the parent and child tables:

```
CREATE TABLE attributes (
    attribute_id   INTEGER PRIMARY KEY,
    attribute_name TEXT NOT NULL UNIQUE
);

CREATE TABLE contacts (
    contact_id   INTEGER PRIMARY KEY,
    account_name TEXT NOT NULL
);

CREATE TABLE data (
    data_id    INTEGER PRIMARY KEY,
```

```
    contact_id INTEGER NOT NULL,
    attribute_id INTEGER NOT NULL,
    value      TEXT NOT NULL
);
```

The `contact_id` and `attribute_id` columns link each value to a contact and an attribute. Later queries will use these matching identifiers to combine rows from the three tables.

The `account_name` value makes the contact's source easy to recognize. A larger production database might place account details in a separate table, but that extra relationship is not needed for this lesson.

Inspect the completed definitions:

```
.schema contacts
.schema data
.schema attributes
```

Step 5: Insert the Attributes

The `attributes` table is a lookup table. It stores each attribute name once and gives it a short integer identifier.

```
INSERT INTO attributes (attribute_id, attribute_name) VALUES
    (1, 'Name'),
    (2, 'Phone'),
    (3, 'Email');
```

Verify the rows:

```
SELECT * FROM attributes ORDER BY attribute_id;
```

```
attribute_id  attribute_name
------------  --------------
1             Name
2             Phone
3             Email
```

Step 6: Insert a Contact and Its Attributes

Insert the contact first, followed by its name, phone number, and email address:

```
INSERT INTO contacts (contact_id, account_name)
VALUES (101, 'Personal');

INSERT INTO data (data_id, contact_id, attribute_id, value) VALUES
    (301, 101, 1, 'John Smith'),
    (302, 101, 2, '+1 555-1234'),
    (303, 101, 3, 'john@gmail.com');
```

All three rows have `contact_id = 101`, so they belong to the same contact. Their different `attribute_id` values identify the values as a name, a phone number, and an email address.

View the stored rows directly:

```
SELECT * FROM contacts;
SELECT * FROM data ORDER BY data_id;
```

```
contact_id  account_name
----------  ------------
101         Personal

data_id  contact_id  attribute_id  value
-------  ----------  ------------  --------------
301      101         1             John Smith
302      101         2             +1 555-1234
303      101         3             john@gmail.com
```

The `value` column alone does not explain what a value means. We must join `data` with `attributes` to interpret it. The column uses SQLite's `TEXT` storage class because names, email addresses, and phone numbers can all contain nonnumeric characters. A phone number should not be stored as an integer because it may contain a plus sign, spaces, or a leading zero.

Step 7: Reconstruct a Readable Contact with JOINs

Join all three tables by following their key relationships:

```sql
SELECT c.contact_id,
       c.account_name,
       a.attribute_name,
       d.value
FROM contacts AS c
JOIN data AS d
  ON d.contact_id = c.contact_id
JOIN attributes AS a
  ON a.attribute_id = d.attribute_id
WHERE c.contact_id = 101
ORDER BY d.data_id;
```

```
contact_id  account_name  attribute_name  value
----------  ------------  --------------  --------------
101         Personal      Name            John Smith
101         Personal      Phone           +1 555-1234
101         Personal      Email           john@gmail.com
```

The first `JOIN` finds all attributes owned by contact 101. The second `JOIN` converts each numeric `attribute_id` into a readable attribute name.

Step 8: Store More Than One Value for an Attribute

The EAV design does not need columns named `phone1` and `phone2`. Add another row for the `Phone` attribute:

```sql
INSERT INTO data (data_id, contact_id, attribute_id, value)
VALUES (304, 101, 2, '+1 555-9876');
```

Retrieve all of John's phone numbers:

```sql
SELECT d.data_id, d.value AS phone
FROM data AS d
```

```
JOIN attributes AS a
  ON a.attribute_id = d.attribute_id
WHERE d.contact_id = 101
  AND a.attribute_name = 'Phone'
ORDER BY d.data_id;
```

```
data_id  phone
-------  -----------
302      +1 555-1234
304      +1 555-9876
```

This is the main benefit of the EAV model: one contact can have any number of attributes without changing the database schema.

Step 9: Useful Queries

Find a contact by part of a stored value:

```
SELECT c.contact_id,
       c.account_name,
       a.attribute_name,
       d.value
FROM data AS d
JOIN contacts AS c ON c.contact_id = d.contact_id
JOIN attributes AS a ON a.attribute_id = d.attribute_id
WHERE lower(d.value) LIKE '%john%';
```

The query finds both values containing `john`, regardless of letter case:

```
contact_id  account_name  attribute_name  value
----------  ------------  --------------  --------------
101         Personal      Name            John Smith
101         Personal      Email           john@gmail.com
```

Count the number of attributes stored for each contact:

```
SELECT c.contact_id,
       c.account_name,
       COUNT(d.data_id) AS attribute_count
FROM contacts AS c
LEFT JOIN data AS d
  ON d.contact_id = c.contact_id
GROUP BY c.contact_id, c.account_name
ORDER BY c.contact_id;
```

After adding the second phone number, contact 101 has four attribute rows:

```
contact_id  account_name  attribute_count
----------  ------------  ---------------
101         Personal      4
```

Find contacts that have no data rows:

```sql
SELECT c.*
FROM contacts AS c
LEFT JOIN data AS d
  ON d.contact_id = c.contact_id
WHERE d.data_id IS NULL;
```

This query returns no rows in the current example because contact 101 has four matching data rows. If a contact were inserted without any attributes, the query would return that contact.

Compare the Two Designs

The wide table is easier for a small, fixed set of attributes, but the EAV design is more flexible when contacts can have different numbers or kinds of attributes.

Question	One wide table	EAV design
Where is a name stored?	In the `name` column	In `data.value` with attribute `Name`
How is a second phone stored?	Add `phone2` or combine values	Insert another `data` row
How is a new attribute added?	Change the table schema	Add an attribute definition and a data row
How is a contact read?	One simple `SELECT`	A `JOIN` reconstructs readable values
Main tradeoff	Simple but rigid	Flexible but requires related tables and joins
Value validation	Each attribute has its own column	All values share one `TEXT` column

Neither design is automatically correct for every application. The important lesson is to recognize the tradeoff. When the number and kind of attributes are fixed, a wide table may be sufficient. When attributes repeat or change over time, EAV rows avoid repeatedly altering the schema. However, the application or later queries must interpret and validate values according to their attributes.

Try It Yourself

Complete the following exercises:

1. Add a new attribute named `Address` with an unused identifier.
2. Add a street address for contact 101 as a new row in `data`.
3. Write a query that returns only email addresses.
4. Add contact 102 under the `Work` account with two phone numbers and one email address.
5. Write a query that counts phone numbers for each contact.
6. Explain why adding a third phone number does not require an `ALTER TABLE` statement in the EAV design.

Summary

The `contacts` table identifies the owner of a set of attributes and provides a readable account name. The `data` table stores each attribute as a separate row, and the `attributes` table describes what each value means. Matching identifier columns connect the tables, while JOINs reconstruct the rows into readable contact information. The EAV design adds phone numbers, email addresses, and other attributes by inserting rows instead of altering the table schema, at the cost of additional joins and attribute-aware validation.

Exit SQLite when you are finished:

```
.exit
```